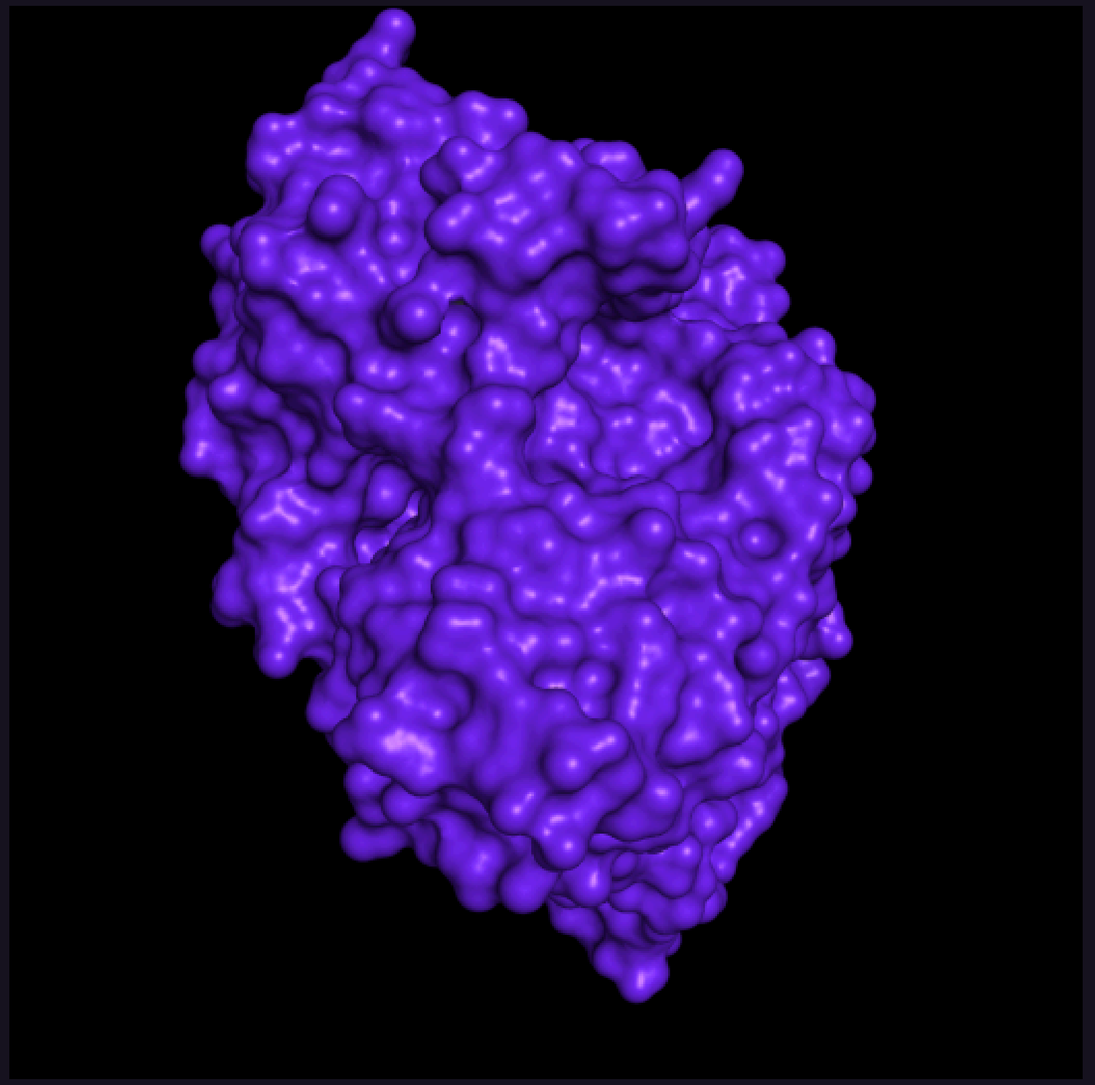
BIO 4525 WORKSHOP

PyMOL scenes and movies

Tell a protein’s story in PyMOL: scenes, camera moves, fades and moving molecules.

github.com/anshuJS/Tutorials_BIOL4525



WHERE THIS STARTED

A 4 minute movie about one enzyme



Watch it on YouTube

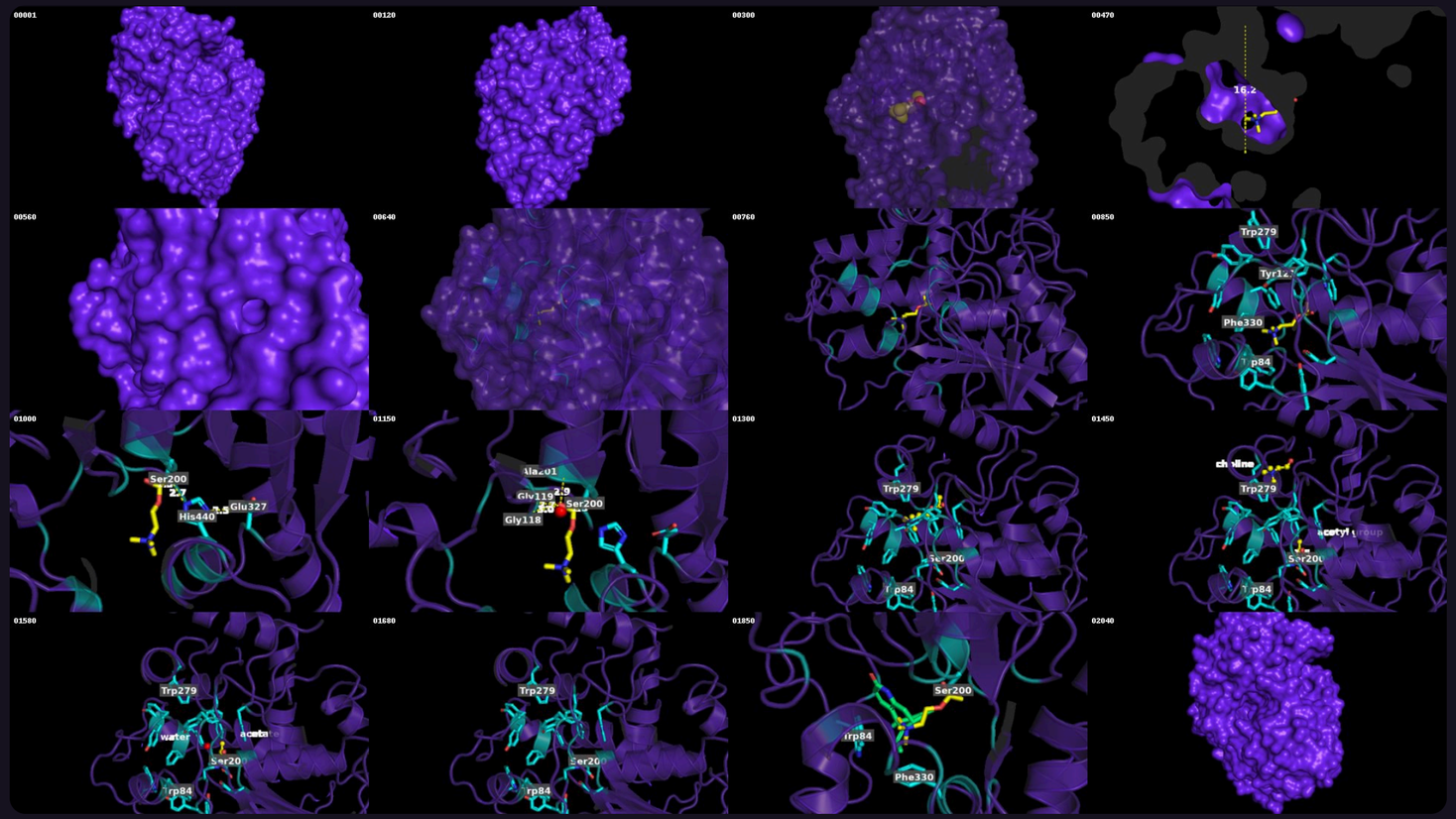
youtu.be/sA3F8CdAZTY

My movie about acetylcholinesterase: PyMOL scenes, plus Blender and video editing.

Today we rebuild its story **using only PyMOL**.

TODAY

The same story, made only in PyMOL



68 seconds, acetylcholinesterase. No other software.

Every shot uses one PyMOL feature you will learn today.

Then you do the same with **your own paper figure**.

THE PLAN

90 minutes, one skill at a time

Minutes	Part	You will
0 to 8	Watch	see the finished movie
8 to 15	Step 1	load and style the protein
15 to 30	Step 2	make scenes
30 to 47	Steps 3 and 4	your first movies: roll, rock, keyframes
47 to 55	Steps 5 and 6	watch the demo: fades and moving molecules
55 to 80	Step 9	turn YOUR paper figure into a movie, in teams
80 to 90	Show and tell	play your movie, save it

SETUP • 3 MIN

Get the files

IN A BROWSER

1. Go to github.com/anshuJS/Tutorials_BIOL4525
2. Green **Code** button, then **Download ZIP**
3. Unzip it on your Desktop

The workshop is the folder **PyMOLMovies** inside **Tutorials_BIOL4525-main**.

THEN TYPE IN PYMOL

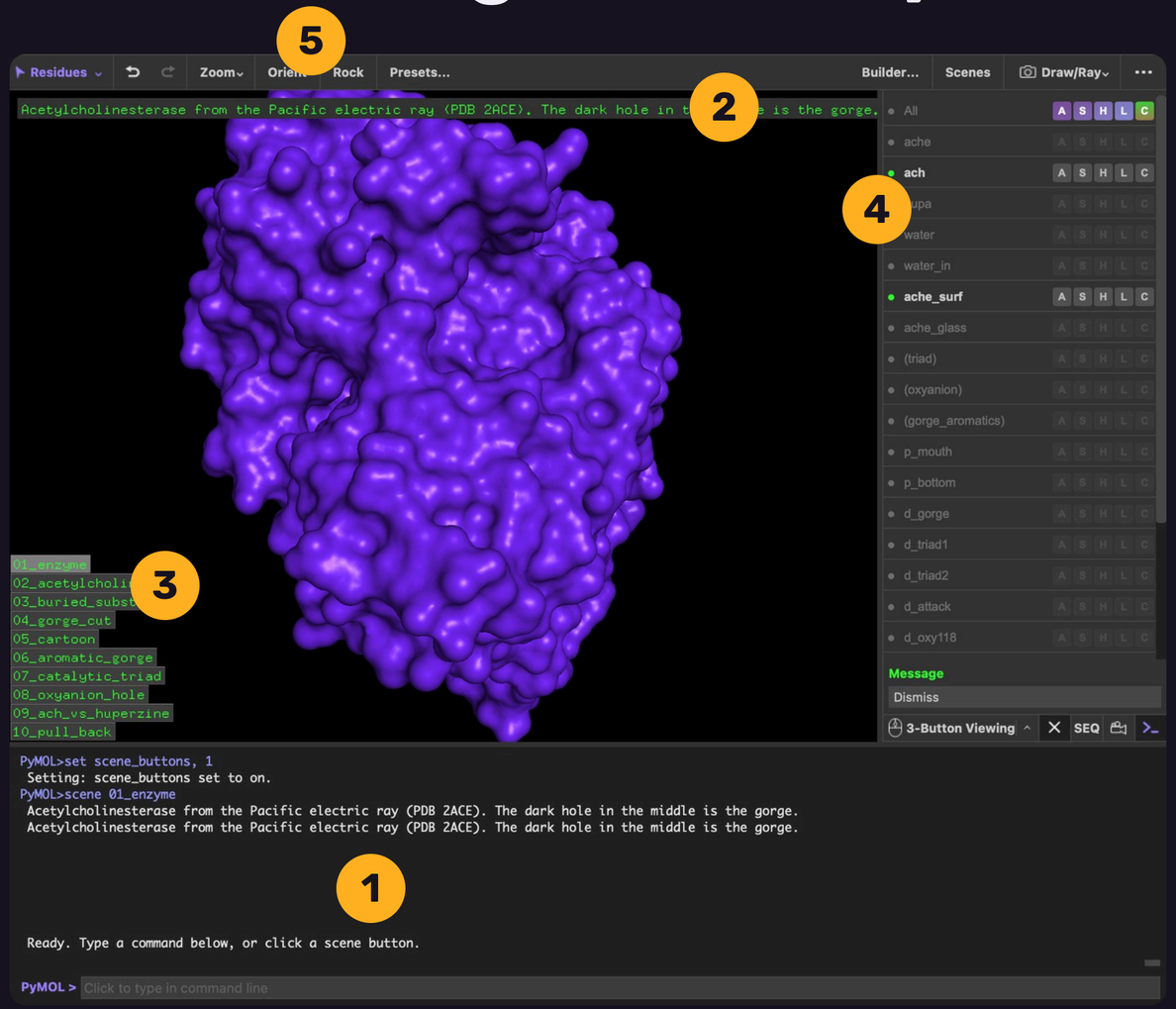
```
cd ~/Desktop/Tutorials_BIOL4525-
main/PyMOLMovies

ls
```

> **Check:** ls lists steps, structures and checkpoints.

Windows: cd C:/Users/YOURNAME/Desktop/Tutorials_BIOL4525-main/PyMOLMovies • Already use git? git clone works too.

Where things are in PyMOL 3.1



1 Command line, at the bottom. Type everything here.

2 Scene caption, across the top of the viewer.

3 Scene buttons. Click one to jump to it.

4 Objects. A green dot means on. Click a name to switch it.

5 Toolbar: Rock, Scenes, Draw/Ray.

Movie controls appear in a **Global Frames** panel at the bottom right once you make a movie.

READING THE CODE

A .pml file is just commands, one per line

```
# a comment. PyMOL skips it
```

Explains the next lines. Never put a ; in one

```
load structures/2ace.pdb, raw2ace
```

Command, then its arguments, separated by commas

```
color cyan, ache and resi 200
```

The last argument is often a **selection**: which atoms

```
show sticks, triad and not name N+C+O
```

Combine with and / or / not. N+C+O means any of them

```
@steps/02_scenes.pml
```

@ runs a whole file, line by line

Selection words you will see: resi (residue number), resn (residue name), name (atom name), chain, within.

STEP 1 • 7 MIN

Load the protein and its actors

TYPE

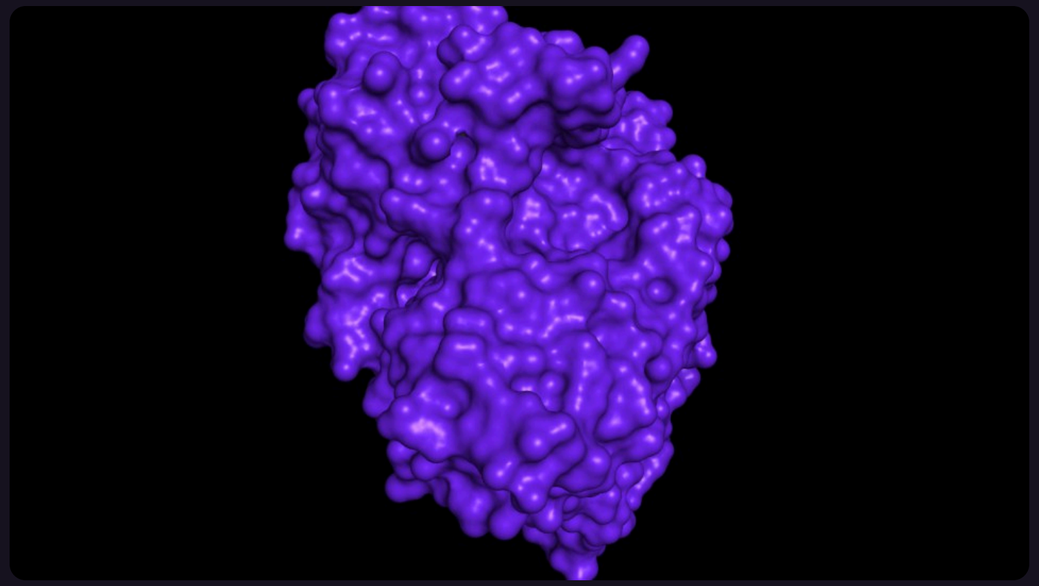
```
@steps/01_load_and_style.pml
```

THEN TRY

Click the object names on the right: **ache**, **ach**, **hupa**, **ache_glass**. Each one is an actor you can switch on and off.

> **Check:** a purple surface, and a long list of objects.

YOU SHOULD SEE



The idea: one object per actor. Separate objects give you control in every later step.

CODE • steps/01_load_and_style.pml • part 1 of 4

Load the two structures

`reinitialize`	Start from an empty PyMOL. Deletes everything
`load structures/2ace.pdb, raw2ace`	Read 2ACE into an object called raw2ace
`load structures/1vot.cif, raw1vot`	Read 1VOT, the enzyme with huperzine A
`align raw1vot and name CA, raw2ace and name CA`	Move 1VOT on top of 2ACE, matching CA atoms
`remove hydro`	Delete hydrogens: they only add clutter here

Check after this part: two objects, raw2ace and raw1vot, sitting on top of each other.

CODE • steps/01_load_and_style.pml • part 2 of 4

Split them into actors

```
create ache, raw2ace and polymer
```

The protein, as its own object

```
create ach, raw2ace and resn ACH
```

The substrate acetylcholine (residue name ACH)

```
create hupa, raw1vot and resn HUP
```

The drug huperzine A, from 1VOT

```
create water, raw2ace and resn HOH and resi 682
```

One crystal water, used in scene 08

```
create water_in, raw2ace and resn HOH and resi 742
```

The water that attacks in step 6

```
delete raw2ace
delete raw1vot
```

The raw files are not needed any more

```
create ache_surf, ache
create ache_glass, ache
```

Two more copies of the protein: one solid surface, one see-through

Why copies? Transparency is a setting, and scenes do not store settings. Two objects = two looks.

CODE • steps/01_load_and_style.pml • part 3 of 4

Name the important residues and colour everything

`select triad, ache and resi 200+440+327`	A named selection: the catalytic triad
`select oxyanion, ache and resi 118+119+201`	The three backbone NH groups of the oxyanion hole
`select gorge_aromatics, ache and resi 70+84+... and not name N+C+O`	14 aromatic residues, side chains only
`bg_color black`	Black background, like the original movie
`set_color ache_purple, [0.45, 0.15, 0.95]`	Invent a colour: red, green, blue from 0 to 1
`color ache_purple, ache or ache_surf or ache_glass`	Everything protein goes purple
`color cyan, ache and elem C and (triad or oxyanion or ...)`	Carbons of the key residues go cyan
`util.cnc ache`	N blue, O red, carbons keep their colour

Selections show in the object panel in brackets, like (triad). They are pointers, not objects.

Surfaces, settings, measurements and labels

`show surface, ache_surf` `show surface, ache_glass`	Surfaces on the two copies
`set transparency, 0.6, ache_glass`	Make one copy see-through (only that object)
`set cartoon_transparency, 0.5, ache`	A see-through cartoon
`set ray_interior_color, grey20`	Colour the inside of a surface you cut open (scene 04)
`pseudoatom p_mouth, ache and resi 279 and name CD2+CE2+...`	An invisible point at the centre of the Trp279 ring
`distance d_gorge, p_mouth, p_bottom`	Dashed line: gorge mouth to bottom, 16 A
`distance d_triad1, ...resi 200 and name OG, ...resi 440 and name NE2`	Ser200 to His440 hydrogen bond, 2.7 A
`pseudoatom lab_triad, ache and resi 200 and not name N+C+O+CA,` `label=Ser200`	A floating label. Scenes forget label text, so each scene gets its own label object

Last lines: disable all, enable ache_surf, orient ache. You start on the purple surface.

STEP 2 • SCENES

A scene is a snapshot you can jump back to

It remembers

- the camera view
- which objects are on
- cartoon, sticks, surface…
- colours
- a caption (message=)

It forgets

- label text
- settings: transparency, sphere size, background

So step 1 made separate label objects and a separate see-through surface.

STEP 2 · BY HAND · 7 MIN

Make two scenes yourself

TYPE, ONE LINE AT A TIME

```
orient ache_surf
scene my_first, store, message=The whole enzyme
disable ache_surf
enable ach
show sticks, ach
zoom ach, 4
scene my_second, store, message=The substrate
```

THEN

Press **Page Up** and **Page Down**, or click the two new scene buttons.

The camera flies, objects switch, the caption changes.

> **Check:** two buttons, my_first and my_second.

Captions: no commas inside message=. The caption stops at the first comma.

STEP 2 • THE STORY • 5 MIN

Load all ten scenes of the story

TYPE

```
@steps/02_scenes.pml
```

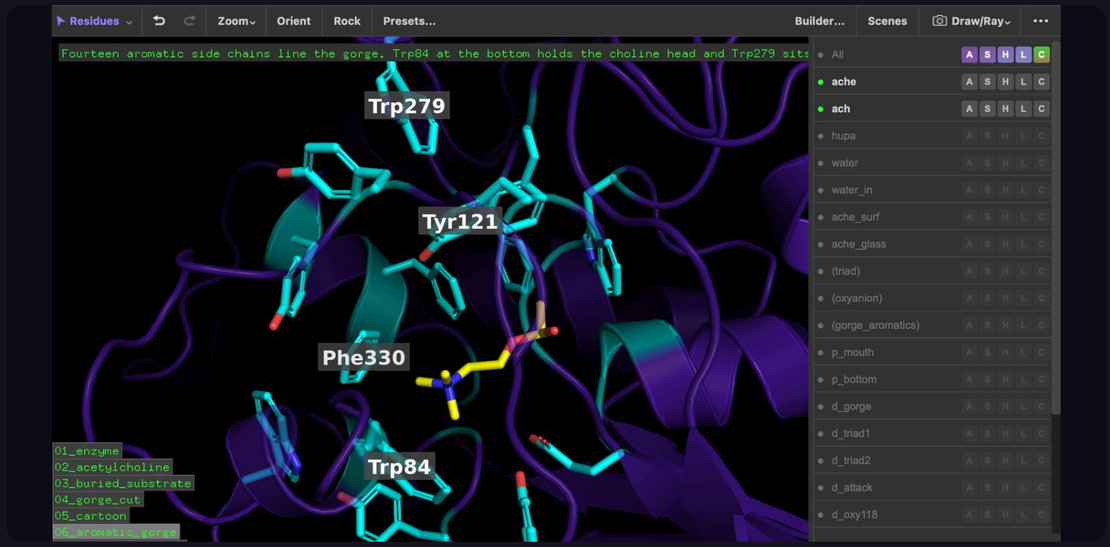
THEN TRY

Click through the scene buttons in order. Read each caption.

```
set scene_animation_duration, 3
```

Check: buttons 01_enzyme to 10_pull_back.

YOU SHOULD SEE



Behind? load checkpoints/after_step2.pse and you are caught up.

INSIDE STEP 2

Every scene is the same four moves

```
# 1 switch off what you do not need
disable all
hide everything, ache or ach or hupa
# 2 switch on and show what you need
enable ache
show cartoon, ache
show sticks, triad and not name N+C+O
enable lab_triad
# 3 place the camera
set_view (... 18 numbers ...)
# 4 store it, with a caption
scene 07_catalytic_triad, store, message=...
```

The 18 numbers in set_view

1 to 9 which way the camera looks

10 to 12 how far away it is (zoom)

13 to 15 the point it orbits around

16 and 17 front and back clipping planes

18 perspective or flat

Never type them. Move with the mouse, type **get_view**, copy what it prints.

CODE • steps/02_scenes.pml • part 1 of 1

What each of the ten scenes switches on

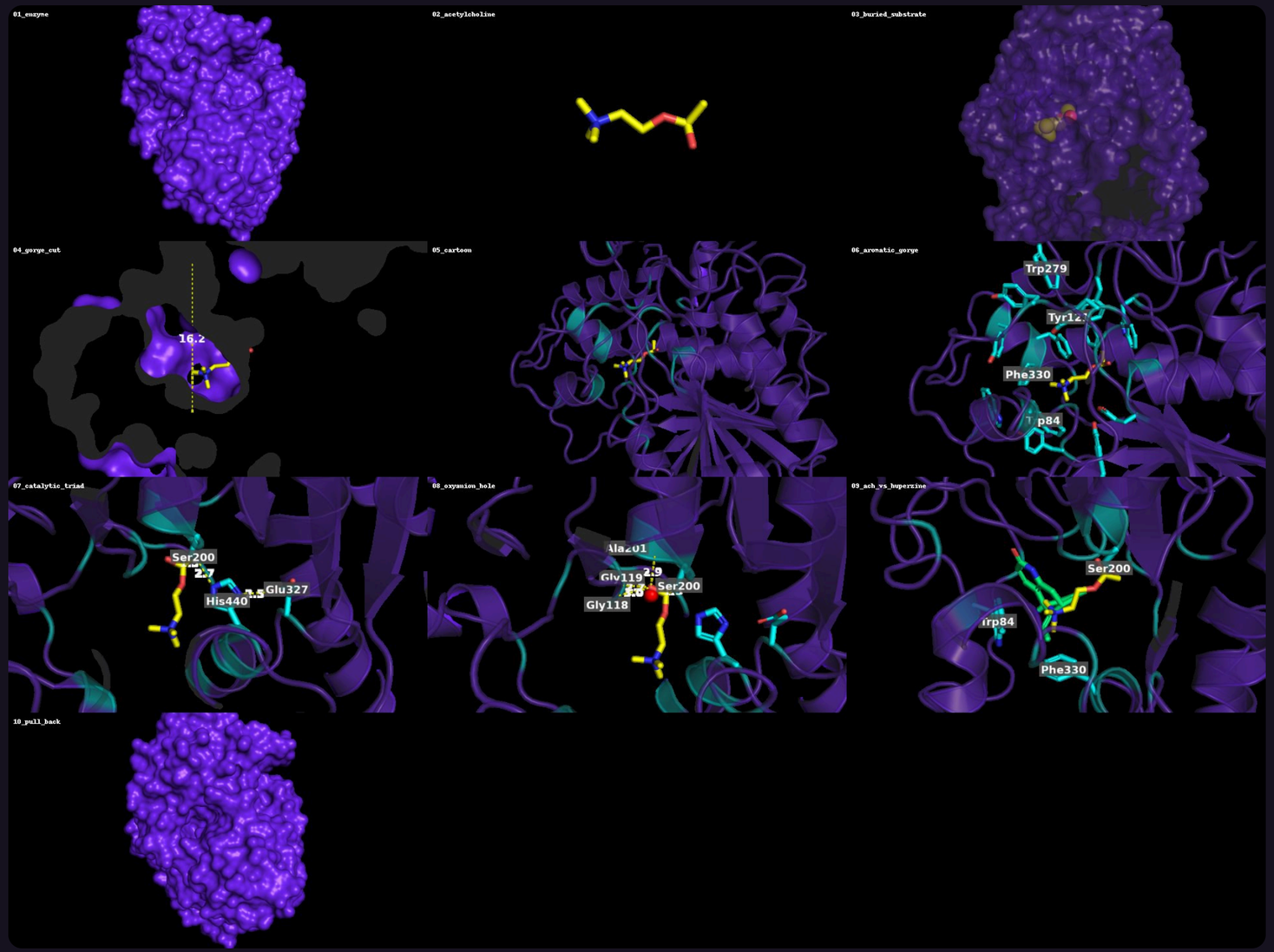
`scene *, clear`	Delete old scenes first
`01_enzyme`	ache_surf + ach sticks. Camera looks into the gorge
`02_acetylcholine`	ach only
`03_buried_substrate`	ache_glass + ach as spheres
`04_gorge_cut`	ache_surf + d_gorge, clip planes only 8 A apart
`05_cartoon / 06_aromatic_gorge`	Cartoon, then + gorge_aromatics sticks + lab_gorge
`07_catalytic_triad / 08_oxyanion_hole`	Triad or oxyanion sticks + their distances + labels
`09_ach_vs_huperzine / 10_pull_back`	ach + hupa + lab_ligands, then the whole surface

Every block is: disable all, hide everything, enable and show, set_view, scene NAME, store, message=...

THE STORY IN TEN SCENES

Wide, closer, the answer, back out

Enzyme, substrate, where it hides, the gorge, the aromatic lining, the catalytic triad, the oxyanion hole, a real drug, pull back.



THE DRUG • PDB 1VOT

How huperzine A blocks the enzyme

1 It takes the substrate’s seat

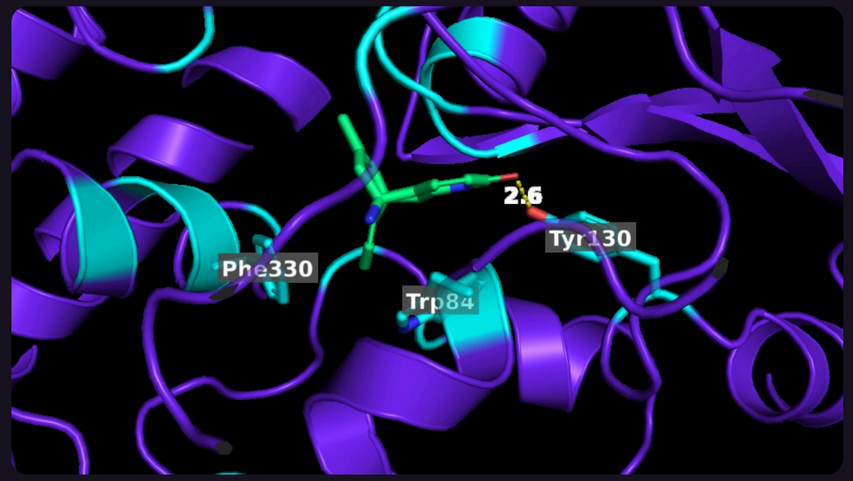
It sits at the bottom of the gorge, exactly where the choline head of acetylcholine must bind. 7 of the 10 ACh atoms would clash with it.

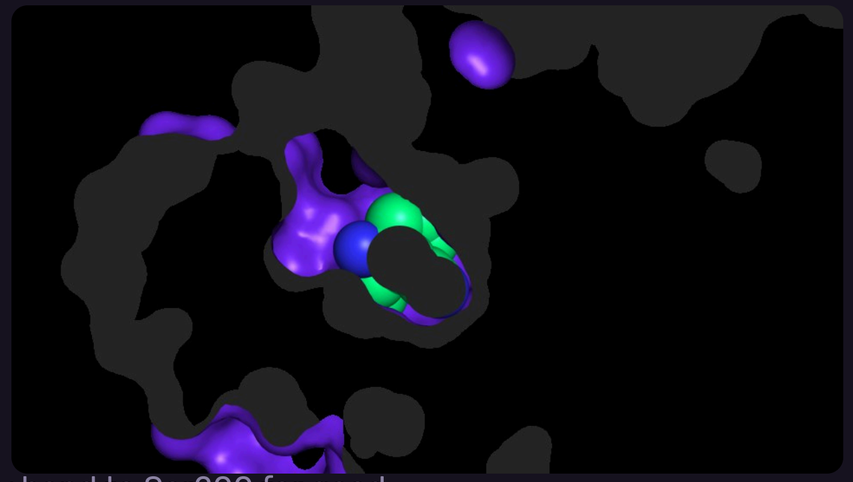
2 Rings and one hydrogen bond hold it

Trp84 (3.9 A) and Phe330 (4.1 A) sandwich it. Its carbonyl oxygen hydrogen bonds Tyr130 (2.6 A).

3 It never bonds the serine

Ser200 is 4.3 A away: no bond. It blocks without reacting, so it can leave again. A **reversible** inhibitor.





Raves et al. 1997, Nat Struct Biol 4:57. Nerve agents and many pesticides do the opposite: they bond to Ser200 for good.

CHALLENGE • 10 MIN • IN PAIRS

Can you show it? One scene for each point

Scene A: it takes the seat

Show hupa and ach together, as sticks, with Trp84 and Phe330.

Scene B: what holds it

Measure the hydrogen bond to Tyr130. Label the three residues.

Scene C: no bond to the serine

Measure from the drug to Ser200. Is it a bond?

COMMANDS YOU WILL NEED

```
enable hupa ach
show sticks, ache and resi 84+330+130 and not name N+C+O
distance d_hb, hupa and name O1, ache and resi 130 and
name OH
distance d_ser, hupa and name C8, ache and resi 200 and
name OG
pseudoatom lab_mine, ache and resi 130 and name OH,
label=Tyr130
scene my_A, store, message=...
```

> **Check:** your two numbers should be 2.6 A and 4.3 A.

Bonus: cut the gorge open like scene 04 and show hupa as spheres. Then join your scenes into a movie with steps/08_your_turn.pml.

STEP 3 · 7 MIN

Your first movie: roll and rock

TYPE

```
@steps/03_roll_and_rock.pml
```

Press **play** in the Global Frames panel (bottom right), or type mplay. Stop with mstop.

THEN A ROCK ON THE TRIAD

```
scene 07_catalytic_triad, animate=0
mset
movie.add_rock(6, angle=30, axis="y", start=1)
```

KNOW THIS

A movie is a list of frames. 30 frames = 1 second.

A camera movie films whatever is on screen. Recall a scene first, then build the movie.

> **Check:** the roll is 240 frames, 8 seconds.

Same as the menu: Movie, Program, Camera Loop. The 2023 AChE video opened with exactly this roll.

CODE • steps/03_roll_and_rock.pml • part 1 of 1

A roll in four lines

```
set movie_fps, 30
```

30 frames per second

```
scene 01_enzyme, animate=0
```

Jump to scene 01 without flying. The roll films this view

```
mset
```

Delete any old movie

```
movie.add_roll(8, axis="y", start=1)
```

Make 240 frames (8 s) turning a full circle about y

```
movie.add_rock(6, angle=30, axis="y", start=1)
```

In the comments: swing 30 degrees back and forth for 6 s

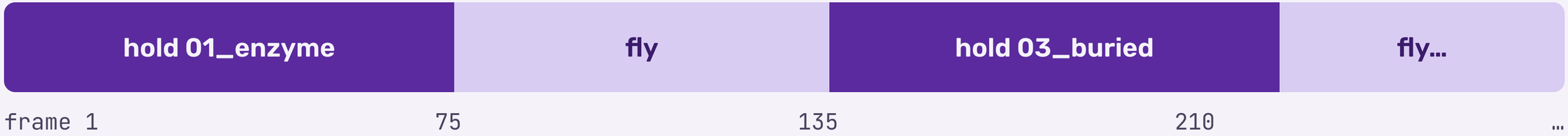
```
mset 1 x240
util.mroll 1, 240, 1
```

The long way: 240 empty frames, then fill them with a turn

movie.add_roll is Python called from the PyMOL prompt. It is the Movie, Program, Camera Loop menu.

STEP 4 • KEYFRAMES

Tell PyMOL a few frames. It fills in the rest.




```
mview store, 1, scene=01_enzyme
mview store, 75, scene=01_enzyme
mview store, 135, scene=03_buried_substrate
mview store, 210, scene=03_buried_substrate
```

Same scene twice = **hold**.

Different scenes = **fly**.

Hold about 2.5 s (75 frames) so people can read.

STEP 4 • 10 MIN

A 43 second movie made of scenes

TYPE, THEN PLAY

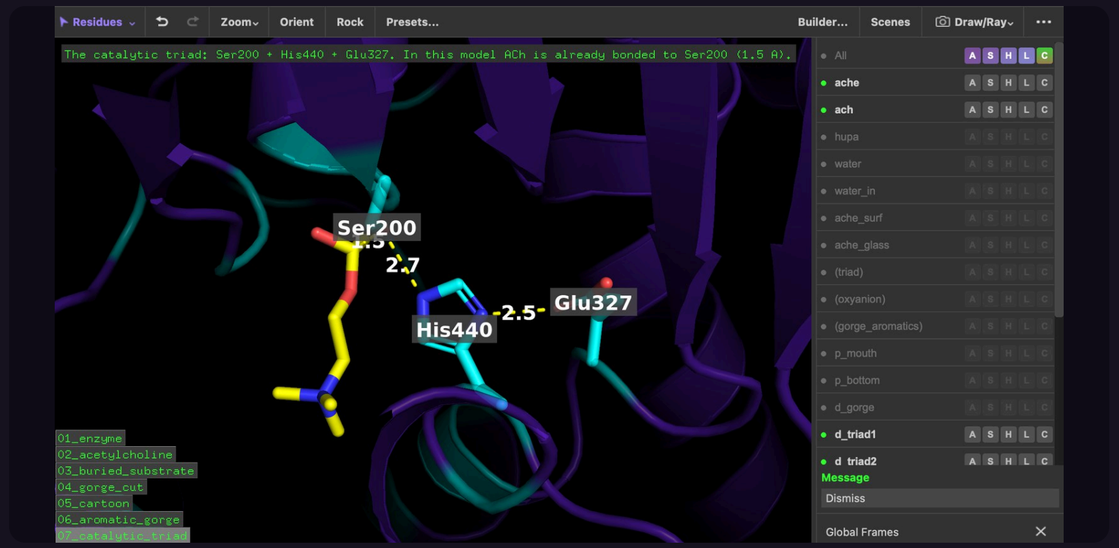
```
@steps/04_scene_movie.pml
```

THEN CHANGE IT

1. Make the triad shot longer
2. Skip a scene: mview clear, 270
3. Add 02_acetylcholine somewhere

> **Check:** the frame counter says 1290.

YOU SHOULD SEE (FRAME 700)



Camera stuck? A movie with keyframes owns the camera. Type mset to delete the movie.

CODE • steps/04_scene_movie.pml • part 1 of 1

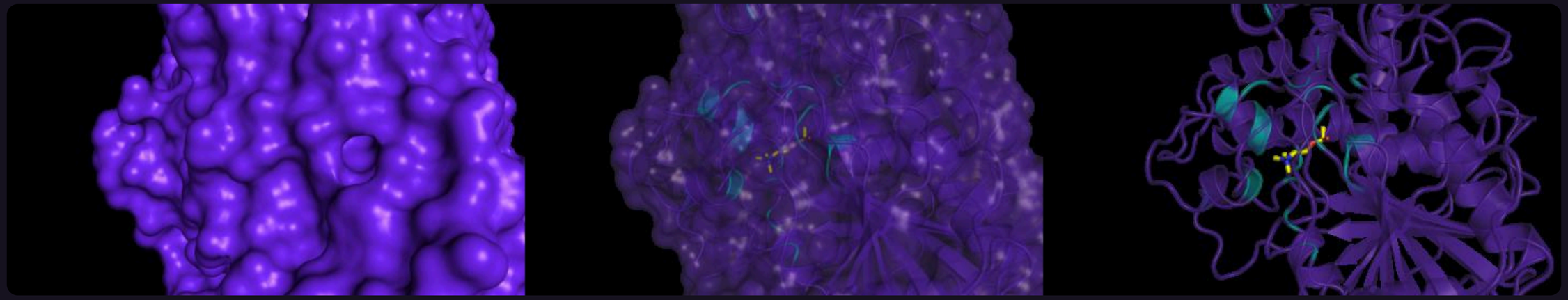
A 43 second movie from the scenes

`mset 1 x1290`	1290 frames = 43 s at 30 fps
`mview store, 1, scene=01_enzyme` `mview store, 75, scene=01_enzyme`	Two keyframes, same scene: hold 2.5 s
`mview store, 135, scene=03_buried_substrate`	Next scene 60 frames later: a 2 s flight
`... the same pattern for 04 to 10 ...`	Hold, fly, hold, fly
`mview store, 1290, scene=10_pull_back`	Last keyframe on the last frame
`frame 1`	Rewind. Press play
`mview clear, 270`	Delete one keyframe (try it)

Seconds x 30 = frames. To lengthen a shot, move its keyframe and every keyframe after it.

STEP 5 • WATCH THE DEMO

Fade the surface away, frame by frame



```
mdo 60: set transparency, 0.3, ache_fade
mdo 61: set transparency, 0.31, ache_fade
… one per frame, written by a short loop
```

Scenes cannot fade: transparency is a setting.

mdo runs a command when the movie reaches a frame.

Try it later: @steps/05_surface_fade.pml • In the 2023 video this shot was made in Blender.

CODE • steps/05_surface_fade.pml • part 1 of 2

Set up the fade shot

```
create ache_fade, ache
```

A fresh copy of the protein, only for this shot

```
hide everything, ache_fade
show surface, ache_fade
```

Surface only

```
set surface_color, ache_purple, ache_fade
```

Purple, like the others

```
scene 05_cartoon, animate=0
enable ache_fade
```

Start from the cartoon scene and lay the surface on top

```
clip slab, 90
```

Open the clipping planes so the whole surface shows

```
mset 1 x180
```

6 seconds

```
frame 1 / mview store
turn y, 25
frame 180 / mview store
```

Camera keyframes: turn 25 degrees over the shot

Nothing fades yet. The next slide adds one command to every frame.

CODE • steps/05_surface_fade.pml • part 2 of 2

Fade it, one frame at a time

```
python
```

Everything until python end is Python, not PyMOL commands

```
for frame_no in range(1, 181):
```

Repeat for frames 1 to 180

```
t = (frame_no - 30) / 90.0
```

0 at frame 30, 1 at frame 120

```
t = min(max(t, 0.0), 1.0)
```

Keep t between 0 and 1

```
cmd.mdo(frame_no, "set transparency, %.3f, ache_fade" % t)
```

Attach a command to this frame: set the transparency to t

```
python end
```

Back to PyMOL commands

```
mappend 121: disable ache_fade
```

Once invisible, switch the surface off (faster)

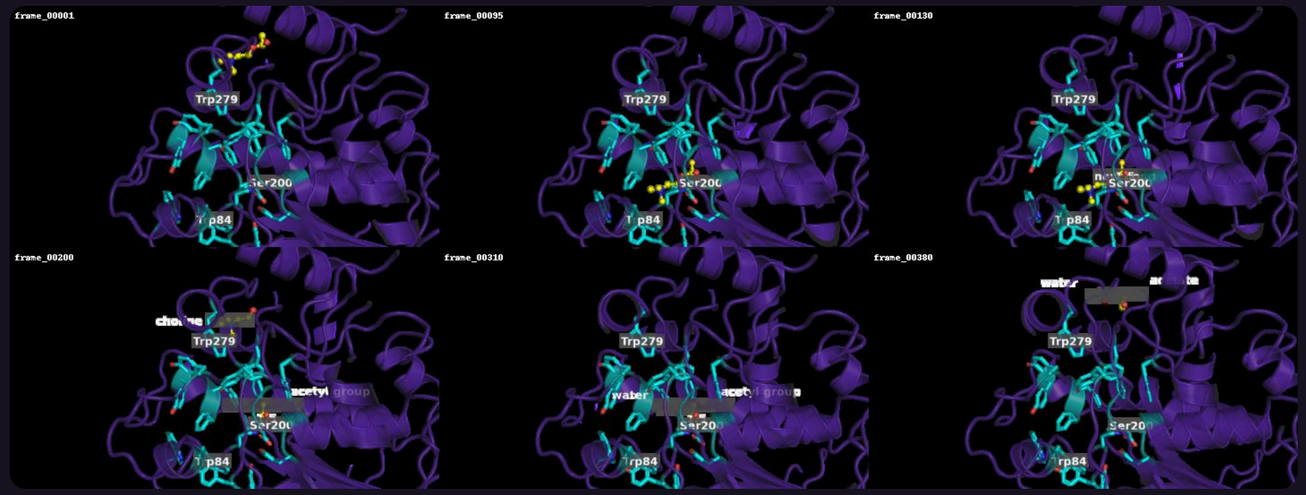
```
mappend 1: enable ache_fade
```

Switch it back on when the movie restarts

In the file the two t lines are one line: t = min(max((frame_no - 30) / 90.0, 0.0), 1.0)

STEP 6 • WATCH THE DEMO

Molecules can have keyframes too



ACh slides down the gorge. Ser200 bonds it. Choline leaves. Water comes in. Acetate leaves.

A cartoon of the chemistry, not a simulation.

```
set movie_auto_store, 0
translate [-6.0, -15.3, 7.3], object=m_ach, camera=0
mview store, 1, object=m_ach start at the gorge mouth
matrix_reset m_ach, mode=1
mview store, 90, object=m_ach arrive in the active site
```

Try it later: @steps/06_mechanism.pml • Also a Blender shot in the 2023 video, now pure PyMOL.

CODE • steps/06_mechanism.pml • part 1 of 3

The actors of the reaction

```
set movie_auto_store, 0
```

Stop PyMOL saving keyframes by itself when objects move

```
create m_ach, ach
```

The whole substrate, for the first part

```
create m_choline, ach and name N1+C2+C3+O4+C8+C9+C10
```

The choline half

```
create m_acetyl, ach and name C5+C6+O7
```

The acetyl half

```
create m_water, water_in
```

The attacking water

```
show sticks / show spheres, m_*
set sphere_scale, 0.25, ...
```

Ball and stick

```
label m_choline and name N1, "choline"
```

A label on an atom moves with its object

```
distance d_newbond, ...resi 200 and name OG, ach and name C5
```

The Ser200 to substrate bond, as a dashed line

Two halves exist from the start. The movie swaps the whole ACh for them at frame 150.

CODE • steps/06_mechanism.pml • part 2 of 3

The set, the camera and the ACh slide

```
show cartoon, ache
show sticks, (gorge_aromatics or triad) and not name N+C+O
```

See-through cartoon with the gorge lining

```
pseudoatom lab_mech, ...resi 200..., label=Ser200
```

Fixed labels: Ser200, Trp84, Trp279

```
set_view (...)
scene 11_mechanism, store, message=...
```

Side view of the gorge, saved as scene 11

```
mset 1 x480
```

16 seconds

```
mview store, 1
turn y, -15
mview store, 480
```

The camera drifts 15 degrees

```
translate [-6.0, -15.3, 7.3], object=m_ach, camera=0
```

Push ACh 18 A up the gorge axis

```
mview store, 1, object=m_ach
```

Remember: at frame 1 ACh is at the mouth

```
matrix_reset m_ach, mode=1
mview store, 90, object=m_ach
```

Back home. At frame 90 it is in the active site

mode=1 resets the movie matrix that translate moved. Plain matrix_reset does not.

CODE • steps/06_mechanism.pml • part 3 of 3

The rest of the reaction, frame by frame

```
mview store, 150 / 240, object=m_choline
```

Choline leaves up the gorge, frames 150 to 240

```
mview store, 240 / 300, object=m_water
```

Water comes down to the acetyl carbon

```
mview store, 330 / 420, object=m_acetyl (and m_water)
```

Acetate leaves, frames 330 to 420

```
mdo 1: enable m_ach
mappend 1: disable m_choline ...
```

Frame 1: only the whole ACh is visible

```
mdo 100: enable d_newbond
```

Frame 100: Ser200 attacks. The new bond appears

```
mdo 150: disable m_ach
mappend 150: enable m_choline
```

Frame 150: swap ACh for its two halves

```
mappend 330: label m_acetyl and name C6, "acetate"
```

Frame 330: relabel as it leaves

```
mdo 421: disable m_acetyl
```

Frame 421: everything has left. Ser200 is free

mdo replaces what a frame does. mappend adds one more command to it.

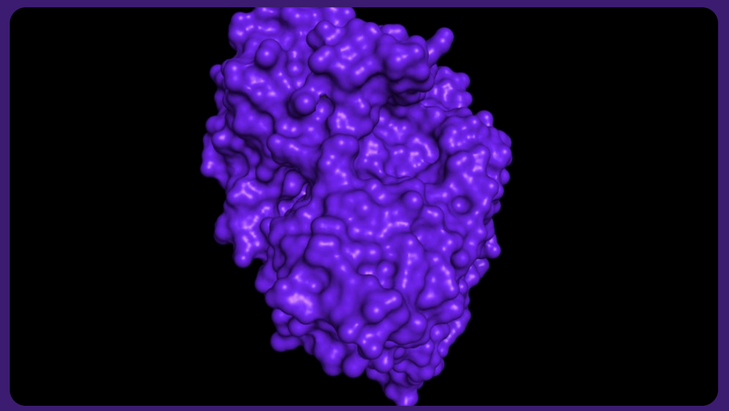
BEFORE YOU START ON YOUR OWN

Things that trip people up

If this happens	Do this
A label or transparency changed in every scene	Scenes forget those: use separate objects
Caption cut short	No commas inside message=
Strange error from a comment line	No ; anywhere in a .pml, even after #
The camera will not stay where you put it	The movie owns it: type mset
A moving object ignores your keyframe	set movie_auto_store, 0 first
Saved movie will not play after loading	Type accept at the security warning

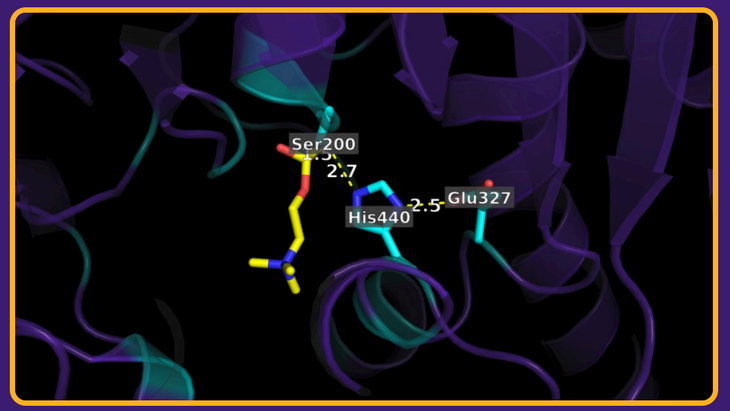
STEP 9 • YOUR TURN • 25 MIN, IN PROJECT TEAMS

Your paper figure script is already a scene



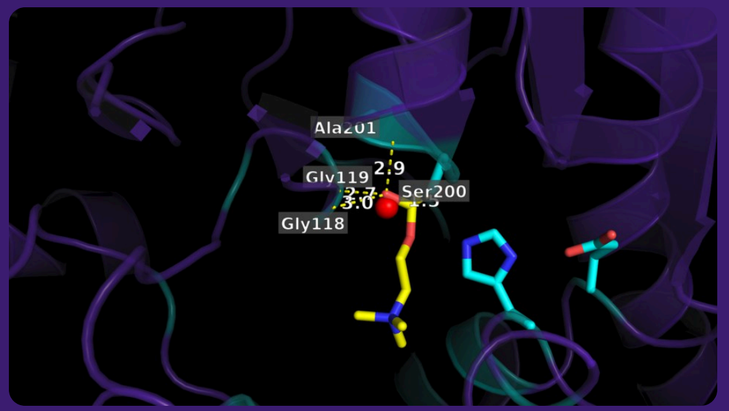
1 Wide

the whole protein



2 Your figure

exactly as in your paper



3 Close-up

what the figure is about, then a gentle rock

12 seconds. Examples shown with AChE. You use your own protein and your own figure script.

STEP 9 • PREPARE • 5 MIN

Get your figure script ready

1. Find the .pml your team uploaded with your figures
2. Copy it. Open the copy in a text editor
3. Put a # in front of every **ray** and **png** line
4. A **load** line points to someone else's laptop? Fix the path, or use **fetch** with the PDB code

BEFORE AND AFTER

```
ray
png figure1.png
becomes
# ray
# png figure1.png
```

Check: run it alone and your figure appears.

Test it on its own first: @path/to/your_figure.pml

STEP 9 • MAKE IT • 20 MIN

Fill in the blanks in step 9

Open **steps/09_your_figure_movie.pml** in a text editor. Fill every ____

```
@____/your_figure.pml
your edited script
message=____
three captions, no commas
zoom ____, 3
small, chain named: chain A and resn NAP
```

THEN

```
@steps/09_your_figure_movie.pml
```

Press play. Change the keyframe numbers until the timing feels right. Run again.

> **Check:** three scene buttons and 390 frames.

A select line in your own script often gives you the close-up for free: zoom pocket, 3

CODE • steps/09_your_figure_movie.pml • part 1 of 2

Your figure becomes three scenes

`reinitialize`	Start clean
`@____/your_figure.pml`	Run your own figure script (ray and png lines commented out)
`scene my_figure, store, message=____`	Store it at once, before touching the mouse
`orient`	Show the whole thing
`scene my_wide, store, message=____`	The wide opening shot
`scene my_figure, animate=0`	Back to your figure
`zoom ____, 3`	Zoom on a small selection, chain named: chain A and resn NAP
`scene my_closeup, store, message=____`	The close-up

Your script already sets colours, representations and set_view. That is a scene.

CODE • steps/09_your_figure_movie.pml • part 2 of 2

Join the three scenes into 12 seconds

```
set movie_fps, 30
mset 1 x300
```

300 frames = 10 s

```
mview store, 1, scene=my_wide
mview store, 45, scene=my_wide
```

Hold the wide shot 1.5 s

```
mview store, 120, scene=my_figure
mview store, 195, scene=my_figure
```

Fly to your figure, hold 2.5 s

```
mview store, 255, scene=my_closeup
mview store, 300, scene=my_closeup
```

Fly to the close-up, hold

```
scene my_closeup, animate=0
movie.add_rock(3, angle=20, axis="y", start=301)
```

Add a 3 s rock after frame 300: 390 frames in all

```
frame 1
```

Rewind and press play

```
save my_figure_movie.pse
```

Save it. Render at home (step 7)

Change the numbers until the timing feels right, then run the file again.

STEP 9 • STRETCH • DONE EARLY?

Two panels, two scenes, one flight

IN A COPY OF PANEL B'S SCRIPT, PUT # IN FRONT OF

- delete all / reinitialize
- fetch or load of something already loaded
- remove lines

Panel B must not undo panel A. Colours are safe: each scene keeps its own.

```
@path/panelA.pml
scene panel_A, store
@path/panelB_edited.pml
scene panel_B, store
mset 1 x240
mview store, 1, scene=panel_A
mview store, 75, scene=panel_A
mview store, 165, scene=panel_B
mview store, 240, scene=panel_B
```

Background colour is a setting: the last script to set it wins for both panels.

SHOW AND TELL • THEN STEP 7

Save first. Render at home.

1 Now

```
save my_figure_movie.pse
```

Keeps every scene and keyframe. You can change the movie next week.

2 Licensed PyMOL

File, Export Movie As, MPEG

Writes .mpg. VLC plays it. MP4 needs ffmpeg.

3 Any PyMOL

```
mpng frames/frame_
```

One PNG per frame. Join them in Fiji or any image-sequence tool.

Each team plays its movie from the front: **30 seconds each**.

A ray-traced 40 s movie can take an hour. Test small first: mode="draw", 640 x 360.

CODE • steps/07_export.pml • part 1 of 1

Save, then export

```
save my_movie.pse
```

The session: objects, scenes, keyframes, mdo. Always first

```
movie.produce("test.mpg", mode="draw", quality=60, width=640,
height=360)
```

A quick, rough test movie

```
movie.produce("my_movie.mpg", mode="ray", quality=90, width=1280,
height=720)
```

The final movie (licensed PyMOL). Slow but clean

```
set ray_trace_frames, 1
viewport 1280, 720
```

Any PyMOL: ray trace every frame at this size

```
mpng frames/frame_
```

Write frames/frame_0001.png, frame_0002.png …

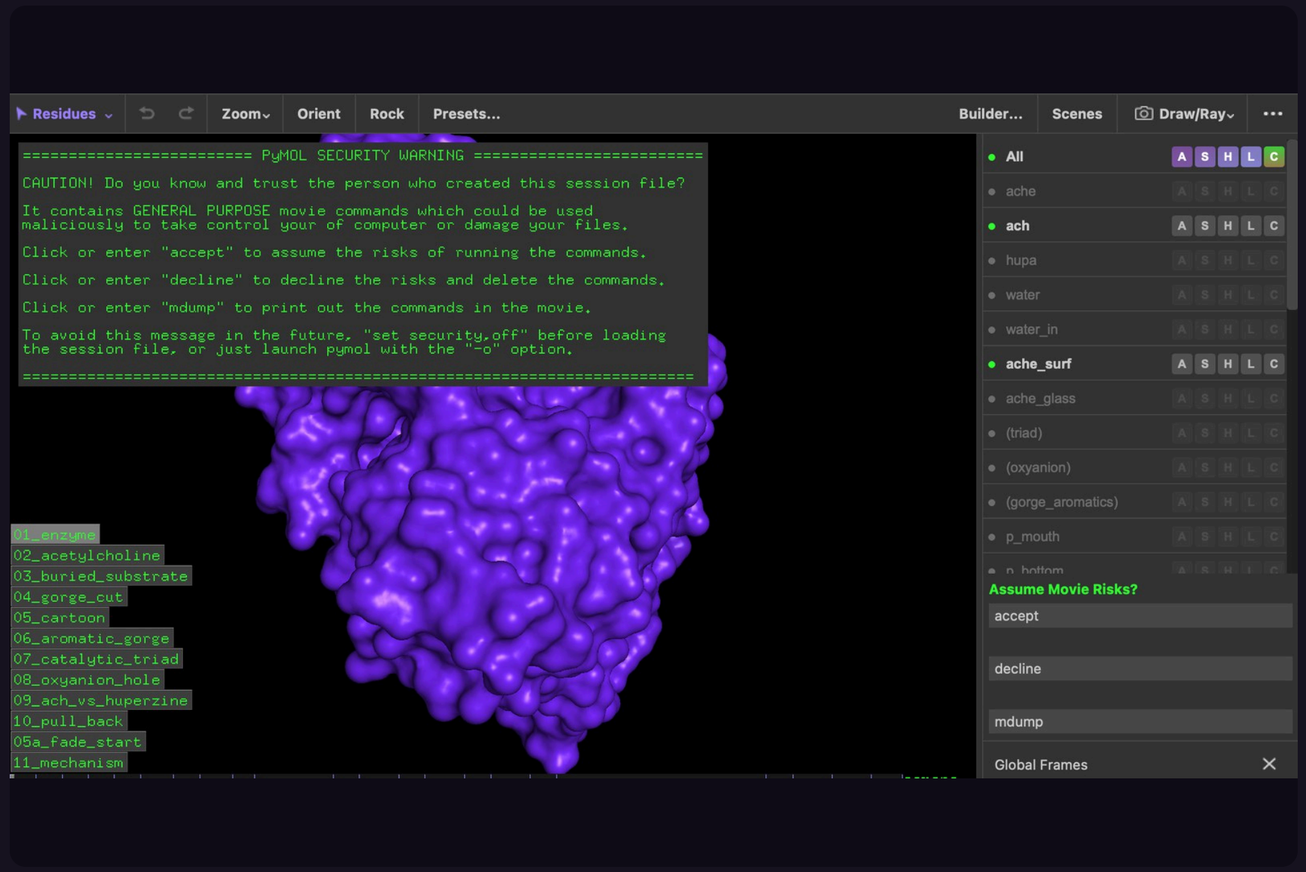
```
png triad.png, width=2400, height=1350, dpi=300, ray=1
```

One still picture for a slide or poster

Everything except save is commented out in the file. Remove the # in front of the one you want.

OPENING A SAVED MOVIE

A security warning is normal. Type accept.



Sessions with **mdo** commands (fades, moving objects) ask first.

Until you accept, the movie will not play.

```
accept
```

Try it: load sessions/AChE_workshop_final.pse, accept, mplay.

CODE • steps/08_your_turn.pml • part 1 of 1

Optional: the huperzine A template

```
disable all
hide everything, ache or ach or hupa
```

Clean slate for each shot

```
enable ____
show ____, ____
```

Fill in: which objects, which representation

```
scene my_01, store, message=____
```

Store the shot with a caption

```
select hupa_site, byres (ache within 4 of hupa)
```

Hint: every residue touching the drug

```
distance d_hupa, hupa and name ____, ache and resi ____ and name ____
```

Hint: one measurement for the answer shot

```
mset 1 x450
mview store, 1, scene=my_01 ...
```

15 s movie: hold, fly, hold, as in step 4

```
save my_huperzine_movie.pse
```

Save it

Question to answer: how does huperzine A stop the enzyme?

Everything is in the repo

`github.com/anshuJS/Tutorials_BIOL4525 › PyMOLMovies`

Handout

every command and the traps, on one page

Optional step 8

a 15 s movie: how does huperzine A block the enzyme?

Your figure movie

a great extra for your lightning talk